НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи № 1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни «Обчислювальні методи»

Виконала: студентка групи ЕАІ-244
Магдич Ольга Валентинівна
Номер у журналі: 22
Варіант: 2
Перевірив: ас. Баранов О.М.

Одеса - 2026

Мета роботи

Навчитися запускати мовні моделі локально через Ollama та з коду через OpenAI-сумісний API, порівняти локальні й хмарні моделі, виміряти затримку та швидкість генерації, оцінити якість за фіксованою рубрикою та дослідити вплив параметрів temperature і top_p.

Конфігурація середовища

Характеристики середовища

Параметр	Значення
Операційна система	Windows 11
Оперативна пам’ять	16 GB
Графіка	Intel Iris Xe Graphics
Python	3.12.2
Git	2.54.0.windows.1
Ollama	0.34.2
Локальні моделі	llama3.2:3b, qwen3:4b
Хмарний провайдер	Groq; openai/gpt-oss-20b

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до qwen3:4b та llama3.2:3b через /api/generate. Спочатку перевірено фактичне питання, потім задачу на міркування з п’ятьма запусками для кожної моделі. Фактичний запит: «Столиця України? Відповідай одним словом.» Задача: «Ручка і зошит разом коштують 110 грн. Ручка на 100 грн дешевша за зошит. Скільки коштує ручка?» Правильна відповідь - 5 грн.

Таблиця 1 - Фактичне питання

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	223	5
Час, с	19,157	8,635
Токенів/с	≈12,00	≈17,94

Таблиця 2 - Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5/5	1/5
Середній eval_count	1263,6	157,2
Середній час, с	157,58	12,12

Обчислення: час = total_duration / 10^9; токенів/с = eval_count / (eval_duration / 10^9). У задачі з ручкою та зошитом x + (x + 100) = 110, звідки x = 5 грн.

Відповіді на питання з кроку 6

1. Я порівнював міркувальну модель (qwen3:4b) та інструктивну (llama3.2:3b) на простому фактичному питанні. Відповідь обидві дали однакову, але міркувальна згенерувала в десятки разів більше токенів. Куди пішли ці токени?

Відповідь: Додаткові токени були витрачені на проміжне міркування та самоперевірку перед короткою кінцевою відповіддю. У таблиці 1 qwen3:4b має 223 згенеровані токени проти 5 у llama3.2:3b; видимі відповіді містять назву того самого міста.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?

Відповідь: Міркування може допомогти у багатокрокових логічних та розрахункових задачах, де потрібно перевіряти проміжні результати. Для

простих фактів його витрати можуть бути невиправданими: тут відповіді про столицю однакові за змістом, але qwen працювала довше.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивні моделі налаштовують на виконання запитів і бажаний формат відповіді; міркувальні додатково навчають будувати та перевіряти багатокрокові рішення, зокрема за допомогою навчання з підкріпленням. Конкретний процес залежить від моделі, і його не можна визначити лише за кількістю токенів у цьому експерименті.

4. Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?

Відповідь: Ні: кількість токенів характеризує обсяг генерації, а не гарантує правильності. У задачі про ручку qwen дала 5 правильних відповідей із 5, але на простому факті додаткові токени не покращили результат.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b ?

Відповідь: У таблиці 1 швидкість qwen становить приблизно 12,00 токенів/с, а llama - 17,94 токенів/с. На цю різницю можуть впливати розмір та архітектура моделей, квантування й використання апаратних ресурсів; наведені дані не дозволяють відокремити внесок кожного чинника.

Висновок за завданням 1

У першому вимірюванні llama швидша на простому факті, а qwen точніша на задачі з міркуванням: 5/5 проти 1/5. Тому додатковий час міркування доцільно витрачати для задач із проміжними обчисленнями, а не автоматично для будь-якого запиту.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Запит варіанта 2: «Поясни різницю між стеком і чергою.» Порівняно system-повідомлення «Ти лаконічний технічний асистент.» і «Відповідай лише українською, максимум двома реченнями.» для обох локальних моделей.

Таблиця 3 - Вплив system-повідомлення

Модель	System	Довжина, символів	Мова	Речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	не зафіксовано	змішана / некоректна	>2	ні
llama3.2:3b	українською, 2 речення	не зафіксовано	частково українська	3	ні
qwen3:4b	лаконічний асистент	≈179	українська	1	так
qwen3:4b	українською, 2 речення	197	українська	3	частково

Довжину відповідей llama для цієї таблиці не вимірювали.

Таблиця 4 - Витрати токенів

Модель	System	prompt_ tokens	completion_ tokens	total_ tokens
llama3.2:3b	лаконічний	51	296	347
llama3.2:3b	українською, 2 речення	55	87	142
qwen3:4b	лаконічний	62	686	748
qwen3:4b	українською, 2 речення	53	1342	1395

Відповіді на питання з кроку 6

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: За таблицею 3 при прямій вимозі «максимум двома реченнями» обидві моделі дали по 3 речення, тому жодна повністю не виконала це обмеження. Одноречennєва відповідь qwen належить іншому system-повідомленню і не є доказом виконання саме цієї вимоги.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: У таблиці 3 зафіксовано, що llama використовувала змішані мовні фрагменти навіть після прямої вказівки відповідати українською.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: У зафіксованих результатах llama показала зростання з 51 до 55, а qwen - зміну з 62 до 53 токенів. Довжина в символах не визначає кількість токенів однозначно; впливають токенізатор і службовий шаблон повідомлень, а для встановлення причин потрібні повні запити.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: qwen3:4b є міркувальною моделлю, тому видимий текст може бути лише частиною згенерованого виводу. Статистика вихідних токенів може враховувати також міркування, залежно від реалізації сервера.

Відповіді на питання з кроку 7

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Моделі відрізняються даними навчання, налаштуванням на виконання інструкцій та здатністю працювати з мовами. System-повідомлення задає бажану поведінку, але невелика модель може порушити навіть прямо

сформульовану вимогу.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення задає загальні правила, а user містить конкретний запит користувача. Пріоритет system підтримується навчанням і шаблоном діалогу, однак це не програмна гарантія виконання кожної інструкції.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Перед видимою відповіддю міркувальна модель може генерувати додаткові проміжні кроки. Через це однакова довжина кінцевого тексту не означає однакових витрат вихідних токенів.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Очікувана поведінка - дотримання system-інструкції, якщо user їй суперечить. Фактична відповідь може порушити пріоритет, тому критичні обмеження слід перевіряти програмно.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Обмежена ємність та особливості багатомовних навчальних даних можуть ускладнювати стабільне виконання мовних інструкцій. Розмір не є єдиною причиною: впливають також якість навчання і конкретний запит.

Власні міркування

System-повідомлення допомагає задати мову, стиль і формат, але не гарантує виконання всіх обмежень. Для критичного формату потрібні програмна перевірка, зрозуміла обробка помилок і повторний запит у разі порушення вимог.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

У compare.py використано однаковий виклик chat.completions.create() для local та cloud. Запит: «Поясни різницю між стеком і чергою. Коротко.» Адреса, ключ і модель задаються у providers.py.

Таблиця 5 - Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-20b
Час відповіді, с (перший / повтор)	18,59 / 24,88	0,86 / 1,09
completion_tokens (перший / повтор)	125 / 232	235 / 298
prompt_tokens (перший / повтор)	44 / 44	90 / 90
total_tokens (перший / повтор)	169 / 276	325 / 388
Довжина відповіді, символів	не зафіксована	не зафіксована
Мова відповіді у повторі	текст українською, але беззмістовний і з мовними дефектами	зрозуміла українська; є окремі граматичні огріхи
Фактичні помилки у повторі	відповідь не пояснює стек і чергу	основне розрізнення LIFO/FIFO правильне

Відповіді на питання з кроку 5

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: У compare.py рядків для переходу змінювати не довелося: цикл послідовно запускає local і cloud через той самий виклик API. Назву моделі й адресу сервера для кожного варіанта задає make_client() у providers.py.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: У повторному запуску хмарна модель відповіла правильно: стек працює за принципом LIFO, черга - FIFO; вона навела операції push/pop та enqueue/dequeue. Локальна модель видала текст про світло й фізичні частинки замість пояснення структур даних, тож хмарна відповідь значно точніша.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так. У локальній відповіді є фактична помилка: вона не пояснює стек і чергу та містить беззмістовний текст про світло. Хмарна відповідь правильно розрізняє LIFO і FIFO; її українська зрозуміла, хоча є окремі граматичні огріхи.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: У повторному вимірюванні локальна відповідь тривала 24,88 с, хмарна - 1,09 с; тобто в цьому запуску хмарна була приблизно у 22,8 раза швидша. Імовірна причина - потужніше серверне обладнання й оптимізація, але один запуск не дає змоги відокремити їхній вплив від мережі та навантаження сервісу.

Відповіді на питання з кроку 6

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: Поширений формат запитів полегшує повторне використання клієнтів та інтеграцій. Підтримка цього інтерфейсу в Ollama дозволяє перемикати провайдера через конфігурацію замість переписування логіки запитів.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: У формулюванні методички згадано 70 млрд параметрів, але в цій роботі використано openai/gpt-oss-20b, а не модель на 70 млрд. Загалом розмір може впливати на якість, однак результат також залежить від архітектури, даних, навчання та виду задачі.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Продуктивне серверне обладнання та оптимізація інференсу можуть переважити витрати на мережу. У таблиці 5 хмарний час менший, але це спостереження не слід поширювати на будь-який провайдер або запит.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Вміст запиту передається зовнішньому сервісу і підпадає під умови його обробки та зберігання. У проєктах ризик зменшують мінімізацією й анонімізацією даних, контролем доступу та перевіркою умов використання сервісу.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit обмежує кількість запитів або токенів за певний час. Розробники працюють у межах цих лімітів через черги, кешування, повтори з паузою та належний тариф, а не через безконтрольні повторні запити.

Власні міркування

Локальну модель оберу для автономної роботи та конфіденційних запитів, якщо відповідь можна перевірити. Хмарна модель у повторному порівнянні швидше пояснила стек і чергу; її варто використовувати, якщо є мережа та можна передавати запит провайдеру.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Код benchmark.py виконує прогрів, потім три послідовні повтори для кожної комбінації та записує медіану кожного показника. Окремі повтори не збережено. Короткий запит - «Столиця Японії?», середній - «Поясни в одному абзаці, що таке рекурсія.», довгий - «Напиши інструкцію з 10 пунктів, як налаштувати робоче середовище розробника.»

У цьому коді символів/с = довжина відповіді / загальний час, включно з TTFT. Це показник наскрізної продуктивності, а не швидкість декодування після першого токена. Медіани показників обчислені незалежно, тому відношення медіан довжини й часу не обов’язково дорівнює медіані швидкості.

Таблиця 7 - Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
local	llama3.2:3b	short	2,113	3,181	11,9
local	llama3.2:3b	medium	2,115	8,85	43,6
local	llama3.2:3b	long	2,148	28,063	40,3
cloud	openai/gpt-oss-20b	short	0,413	0,439	17,8
cloud	openai/gpt-oss-20b	medium	0,395	0,556	922,6
cloud	openai/gpt-oss-20b	long	0,915	1,994	1432,6

Таблиця 8 - Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	7,103	2,254
Всього, с	8,107	4,718

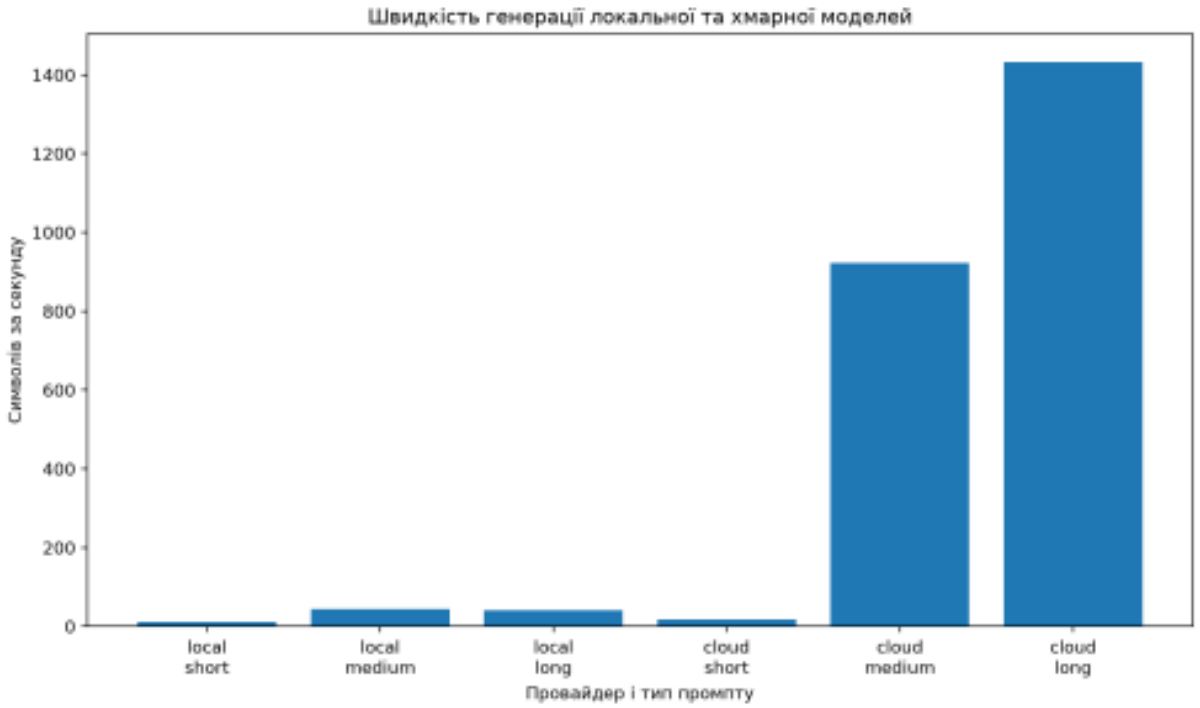


Рисунок 1 - Швидкість для шести комбінацій «модель × промпт»

Відповіді на питання з кроку 6

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: У всіх трьох комбінаціях менший TTFT має хмарна openai/gpt-oss-20b: 0,395-0,915 с проти 2,113-2,148 с у llama. Результат узгоджується з ефективнішою інфраструктурою хмарного запуску, але окремий внесок обладнання та мережі не вимірювався.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Так. У всіх трьох вимірюваннях хмарна модель мала менший TTFT і вищу швидкість: для короткого запиту 0,413 с і 17,8 симв./с проти 2,113 с і 11,9 симв./с у локальної моделі. Результат стосується саме цих запусків.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: Перевага хмарної моделі за швидкістю в символах/с зросла: 17,8/11,9 ≈ 1,5 раза на короткому запиті, 922,6/43,6 ≈ 21,2 раза на середньому та

1432,6/40,3 ≈ 35,5 раза на довгому. За загальним часом хмарна відповідала швидше в усіх випадках; її перевага становила приблизно 7,2 раза на короткому й 14,1 раза на довгому запиті.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Медіана трьох повторів зменшує вплив випадково дуже швидкого або повільного запуску, тож одного запуску недостатньо, щоб оцінити розкид і надійність вимірювання. Окремі результати повторів не збережені, тому конкретну різницю між ними тут визначити неможливо.

5. Чому холодний старт повільніший за теплий?

Відповідь: Холодний старт включає завантаження й підготовку моделі, а теплий використовує вже підготовлений стан. Зафіксований TTFT становив 7,103 с і 2,254 с відповідно.

Відповіді на питання з кроку 7

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: До появи першого видимого фрагмента відбуваються передавання й обробка запиту, підготовка моделі та обробка вхідного контексту; у міркувальної моделі може додаватися приховане міркування. TTFT вимірюють окремо, бо він характеризує початкове очікування, а швидкість подальшого виводу - іншу частину затримки.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Мережа й очікування сервера можуть збільшити початкову затримку. Після початку виводу продуктивне обладнання може генерувати текст швидше; у наших даних хмарний TTFT, навпаки, був кращим.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: При холодному старті модель ще не завантажена та не підготовлена до інференсу. Наступні запуски можуть повторно використовувати її в пам’яті, тому початкова затримка зменшується.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Медіана менше залежить від одного аномально повільного або швидкого запуску, ніж одиничне значення. Кілька повторів зменшують вплив випадкового шуму; окремі значення повторів у цій роботі не аналізувалися.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Токени вже відомого вхідного тексту можна обробляти паралельно під час prefill. Генерація є авторегресивною: кожен наступний токен залежить від попередньо згенерованих, що створює послідовний компонент обчислень.

Власні міркування

Для інтерактивного чат-бота важливий TTFT, бо користувач очікує першого видимого тексту. Для фонової обробки важливі загальний час і наскрізна продуктивність; у цих вимірюваннях хмарна модель мала кращі показники на всіх трьох довжинах запиту.

Завдання 5. Оцінювання якості моделей за рубрикою

Тема варіанта 2 - транспорт. Перевірено відповіді на запити про метро, JSON, стиснення тексту, функцію total_weight(items), задачу про 120 ящиків і перевезення вантажів. За офіційним джерелом [2] метро відкрито 6 листопада 1960 року; станом на 2020 рік воно мало 3 лінії.

Запит для перевірки стискання

Стиснути до двох речень, зберегти всі числа. Вихідний текст: «Київський метрополітен відкрили 6 листопада 1960 року. Перша черга складалася з 5

станцій. Її довжина становила 5,2 кілометра. Київський метрополітен став третім у СРСР після Москви та Ленінграда. На момент публікації матеріалу метро мало 3 лінії загальною довжиною майже 70 кілометрів. Кількість діючих станцій становила 52».

Таблиця 13 - Оцінка якості за рубрикою (прогін 1 / прогін 2)

Завдання	llama3.2:3b	qwen3:4b	Хмарна
1. Факт	0 / 0	0 / 0	2 / 2
2. Формат	2 / 2	2 / 2	2 / 2
3. Стиснення	0 / 0	0 / 0	2 / 2
4. Код	2 / 2	2 / 2	2 / 1
5. Логіка	0 / 0	2 / 2	2 / 2
6. Українська	0 / 0	0 / 1	0 / 1
Разом, балів /12	4 / 4	6 / 7	10 / 10

У кожній клітинці вказано бали двох оцінених відповідей. Для кожної моделі та критерію використано однакову кількість прогонів.

Таблиця 14 - Розбіжність оцінок між двома прогонами

Завдання	llama3.2:3b	qwen3:4b	Хмарна
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	так: у другому прогоні markdown-обгортка
5. Логіка	ні	ні	ні

6. Українська	ні	так	так

Перевірка відповідей за критеріями

Формат: json.loads() успішно розібрав усі 6 JSON-відповідей без змін. Значення всередині JSON не є правильними автоматично: llama в першому прогоні дала рік 1966, qwen у другому - 2019, а кількість ліній у цих відповідях відрізняється від 3 у джерелі [2].

Стиснення: llama втратила частину чисел і отримала 0 балів у двох прогонах. Хмарна модель зберегла числа без спотворень у двох реченнях і отримала 2 бали в обох прогонах. Qwen також використала два речення, але замінила «майже 70 кілометрів» на «70 кілометрів», подавши приблизне значення як точне; за критерієм «усі числа збережені без спотворень» це 0 балів у двох прогонах.

Код: функції перевірено на [], [{weight: 2}] і [{weight: 2}, {weight: 3.5}]; очікувані суми - 0, 2 і 5,5. Усі шість нових функцій синтаксично правильні та проходять ці тести. Виняток за форматуванням - друга хмарна відповідь: код загорнутий у markdown, тому за рубрикою це часткове виконання (1 бал).

Мова і зміст: перша відповідь qwen стверджує, що навалочні перевезення призначені для малих вантажів і коротких шляхів; ця умова не є коректним визначенням. Перша хмарна відповідь підміняє навалочне перевезення roll-on/roll-off. Тому оцінку за зміст слід визначати з урахуванням цих помилок, а не лише граматики.

Відповіді на питання з кроку 6

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: За двома прогонами хмарна модель набрала 10/12 в обох; qwen3:4b - 6/12 і 7/12; llama3.2:3b - 4/12 в обох. Основні відмінності: llama помилилася у факті, втратила числа під час стискання та не розв’язала задачу на логіку;

локальні моделі помилилися у фактичному запиті, а хмарна в одному прогоні обгорнула код у markdown.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: Оцінки розійшлися для хмарної моделі у завданні з кодом та для qwen і хмарної моделі у завданні про українську мову. Це показує, що одна відповідь не завжди відображає стабільну якість; бали треба читати разом із конкретними відповідями.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так: усі шість JSON-відповідей двох прогонів розбираються через json.loads() без виправлень. Водночас парсинг не підтверджує істинність значень: моделі вказували неправильні роки та кількість ліній.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так: у llama є беззмістовні конструкції та дефекти мови; у qwen - русизми й граматичні помилки. Хмарна відповідь була зрозумілішою, але в першому прогоні переплутала навалочне перевезення з roll-on/roll-off.

Приклад провалу з кроку 7

Повний запит: «120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів? Покажи хід розрахунку.»

Повна відповідь llama3.2:3b з першого локального прогону Хід розрахунку:

1. Переведем данную інформацію в грамми:

- 120 ящиків по 8 кг: 120 x 8000 = 960 000 г

- Решта по 12 кг: 120 x 12000 = 1 440 000 г

- Общая вантажопідйомність вантажівки: 1200 000 г

2. Упустимий вантаж: 960 000 + 1 440 000 = 2 400 000 г

3. Решаем, сколько рейсів необхідно, для перенося всього вантажа:

- Общий вантаж: 1200 000 г

- Упустимий вантаж за рейс: 2 400 000 г

- Втратим вантаж на рейс: 1200 000 ÷ 2400 000 = 0,5

4. Втратим вантаж на рейс: 0,5, що означає, що на рейс приходиться 0,5 рейсу.
Правильна відповідь: 120 - 45 = 75 ящиків по 12 кг; загальна маса 45 × 8 + 75 × 12 = 1260 кг. Потрібно 2 рейси: наприклад, 45 ящиків по 8 кг і 70 по 12 кг дають 1200 кг, а решта 5 по 12 кг - 60 кг.

Пояснення: модель двічі використала загальну кількість 120, порахувавши 120 ящиків по 8 кг і ще 120 по 12 кг замість двох груп по 45 та 75. Потім вона поділила вантажопідйомність на помилкову масу у зворотному порядку; правдоподібне оформлення кроків не підтверджує правильність міркування.

Відповіді на питання з кроку 8

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Використовують автоматичні тести з еталонами, перевірку формату та виконання згенерованого коду на тестових прикладах. Для відкритих відповідей додають оцінювання людьми або моделлю-суддею, перевіряючи узгодженість оцінок і відповідність реальному завданню.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Модель може відтворювати поширені у навчальних прикладах markdown-обгортки, навіть коли запит просить їх не використовувати. Для автоматичної обробки необхідні парсер і перевірка схеми; у цьому експерименті JSON-відповіді були без обгортки.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: На якість впливають обсяг і якість україномовних даних, токенізація

та налаштування моделі. Невелика ємність може посилювати проблему, але сам розмір не визначає рівень української мови однозначно.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: При ненульовій temperature вибір токенів може бути випадковим, тому повтори здатні змінити зміст і якість. Крім того, впливають числові особливості інференсу; оцінювати надійність треба на кількох збережених відповідях.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація - правдоподібний, але фактично неправильний згенерований зміст. Побудова ймовірного продовження тексту не є перевіркою істинності, тому без належного навчання, джерел і валідації модель може впевнено вигадувати факти.

Власні міркування

Рубрика дозволяє порівнювати конкретні типи поведінки, але її потрібно застосовувати до тих самих запитів і відповідей. Найпоказовіші перевірки тут - втрати чисел, логічна задача й тест коду на порожньому списку. Для українськомовного тексту я розглядатиму хмарну модель або qwen з перевіркою фактів і редактурою; загальний рейтинг потребує додаткової перевірки на інших прикладах.

Завдання 6. Керування випадковістю: temperature і top_p

Локальна модель - llama3.2:3b. Творчий запит: «Придумай назву застосунку нагадувань про ліки.» Фактичний запит: «У якому році було відкрито Київський метрополітен?» У серії top_p температуру зафіксовано на 0,7. Еталон факту - 1960 рік [2].

Таблиця 16 - Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0,0	1	5
temperature	0,7	5	4
temperature	1,5	5	0*
top_p	0,1	2	5
top_p	0,5	5	5
top_p	1,0	5	5

* Одна відповідь при temperature=1,5 містила число 1960, але також вигаданий «метромосковий ремонтно-новаторський план» і беззмістовні твердження. Її не зараховано як правильну лише через збіг року.

Таблиця 17 - Перевірка відтворюваності фактичного запиту

Показник	Значення
Кількість запусків	10
temperature	0,0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так
Правильних відповідей	10 із 10

Для перевірки повторюваності я поставила фактичне запитання десять разів при temperature=0. Усі відповіді були однакові й правильні: «Київський метрополітен було відкрито в 1960 році.»

Відповіді на питання з кроку 7

1. Яке значення температури ви поставите агенту, який має повертати

JSON, і чому?

Відповідь: Для агента, який повертає JSON, оберу temperature=0 і програмну перевірку формату та значень. Це зменшує випадковість, але не гарантує правильність кожної відповіді.

2. Яке — генератору варіантів заголовків?

Відповідь: Для генератора заголовків почну з temperature=0,7: у творчому експерименті вона дала 5 унікальних відповідей із 5. Підвищення до 1,5 не збільшило цей показник, натомість у фактичній задачі спричинило тяжкі дефекти.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Temperature змінює співвідношення ймовірностей кандидатів через масштабування логітів. Top_p обмежує вибір найімовірнішим набором токенів до заданої сумарної ймовірності.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Ні, temperature=0 не дає універсальної гарантії однакового результату. У десяти повторах фактичного запитання всі відповіді дослівно збіглися, тож отримано 1 унікальну відповідь.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з’явитися.

Відповідь: Показова відповідь: «У 2014 році Київський метрополітет відквався.» Правильний рік - 1960; висока температура збільшує ймовірність менш імовірних токенів, що тут призвело і до помилки факту, і до мовних дефектів.

Відповіді на питання з кроку 8

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Для T>0 temperature масштабує логіти: $p_i = \exp(z_i/T) / \Sigma_j$

exp(z_j/T). Менше T концентрує розподіл, більше робить його пласкішим; при T=0 реалізація зазвичай переходить до вибору найімовірнішого токена.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: Top_p залишає найменший набір найімовірніших токенів із сумарною ймовірністю не меншою за p та перенормовує його. Temperature змінює самі відносні ймовірності, тому механізми різні.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Одночасна зміна можлива, але для зрозумілого експерименту краще спочатку змінювати один параметр. У серії top_p температуру було зафіксовано на 0,7, що дозволяє відокремити вплив відсікання кандидатів.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Числові похибки, паралельні обчислення та відмінності реалізації можуть змінювати результат навіть при виборі найімовірнішого токена. У наших десяти повторах відмінностей не було, але це спостереження для конкретної моделі й запуску, а не загальна гарантія.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для структурованих даних доцільно почати з 0 або близького до нуля значення і перевіряти вихід програмно. Для творчого тексту можна почати з 0,7 та оцінити якість і різноманітність; універсального значення для всіх моделей немає.

Власні міркування

Для наступних лабораторних зі структурованими даними оберу temperature=0 і перевірку JSON та фактичних значень. Для творчих варіантів почну з 0,7. У цьому експерименті збільшення температури погіршило точність відповіді на

фактичне запитання, але збільшило різноманітність назв.

Підсумкове порівняння

Таблиця 18 - Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B)	Хмарна
Модель	llama3.2:3b	qwen3:4b	openai/gpt-oss-20b
Швидкість генерації 11,9-43,6 симв./с за симв./с за	таблицею 7	≈12 ток./с у простому тесті; наскрізні симв./с не виміряно	17,8-1432,6 симв./с за таблицею 7
TTFT	2,113-2,148 с	окремо не вимірювався	0,395-0,915 с
Якість, два прогони /12	4/12; 4/12	6/12; 7/12	10/12; 10/12
Стабільність формату (завдання 2)	2 із 2 JSON парсяться без змін; значення можуть бути хибними	2 із 2 JSON парсяться без змін; значення можуть бути хибними	2 із 2 JSON парсяться без змін; значення можуть бути хибними
Потрібні ресурси	локальний ПК, RAM для моделі та контексту	більше локальних ресурсів; фактичний обсяг не виміряно	обладнання провайдера, клієнтський ПК та інтернет
Вартість	без оплати API; витрати на обладнання й електроенергію	без оплати API; витрати на обладнання й електроенергію	залежить від тарифу та лімітів; вартість запусків не зафіксовано
Куди йдуть ваші дані	обробка локально при localhost	обробка локально при localhost	запити передаються зовнішньому провайдеру

Працює без інтернету	так, після завантаження моделі	так, після завантаження моделі	ні
Коли обирати саме її	Коли потрібні приватність і робота без інтернету та завдання прості; результати треба перевіряти.	Коли потрібно локально розв’язувати багатокрокові задачі й комп’ютер має достатньо ресурсів; правильність відповідей усе одно треба перевіряти.	Коли важливі швидкий відгук і продуктивність, є інтернет і дозволено передавати дані провайдеру; факти й формат слід перевіряти.

Оцінки в рядку якості подані окремо для двох прогонів; вони не є гарантією результату на інших запитах. Швидкість у токенах/с та символах/с отримана в різних експериментах і не є прямим порівнянням однакових одиниць.

Загальні висновки

Локальні й хмарні моделі використовують однакову структуру клієнтського виклику; відмінності зібрані в конфігурації провайдера. За наданими результатами таблиці 7 хмарна модель мала нижчий TTFT, менший загальний час і вищу швидкість у символах/с на всіх трьох запитах. Це спостереження стосується лише зафіксованих вимірювань.

За рубрикою хмарна модель отримала 10/12 в обох прогонах, qwen - 6/12 і 7/12, llama - 4/12 в обох. Локальні моделі помилилися у фактичному питанні; llama втратила числа під час стискання та помилилася в логічній задачі, а qwen змінила приблизне значення на точне. Цей результат стосується перевірених запитів.

Коректний JSON не гарантує коректних даних. System-повідомлення та низька температура допомагають керувати поведінкою, але не замінюють парсер, тести

й перевірку фактів. У десяти повторах фактичного запиту при temperature=0 отримано однакову правильну відповідь; при 1,5 усі п’ять відповідей містили помилки.

Модель слід обирати за умовами роботи, потрібною швидкістю, приватністю й якістю саме на своїх запитах. Порівняння показує перевагу хмарної моделі в цьому наборі завдань, але для важливих відповідей потрібні перевірка фактів і тестування формату.

Джерела та експериментальні матеріали

[1] Лабораторна робота № 1 «Розгортання середовища та порівняльний аналіз мовних моделей». Методичні вказівки, 29 сторінок.

[2] Голосіївська районна в місті Києві державна адміністрація. «Сьогодні столичному метрополітену виповнилось 60 років!», 06.11.2020. https://golos.kyivcity.gov.ua/news/sogodni-stolichnomu-metropolitenu-vipovnilos-60-rokiv (перевірено 13.10.2026).

[3] Ollama. Thinking. https://docs.ollama.com/capabilities/thinking (перевірено 13.10.2026).

[4] Hugging Face. Generation. https://huggingface.co/docs/transformers/main/en/main_classes/text_generation (перевірено 13.10.2026).

[5] Звіт з лабораторної роботи № 1, Pull Request № 19. https://github.com/oleksii-m-baranov/computational-methods-2026-practice/pull/19.